答信 核心架构设计（**v0.1**）

银行智能客服回答的第三方验收与分级放行：验证内核的结构、不变量、接口与实现路径

编制日期：2026-11-01　　状态：工作稿

本文不涉及任何真实机构。所有示例均为虚构；对外材料一律使用机构代号，不点名。

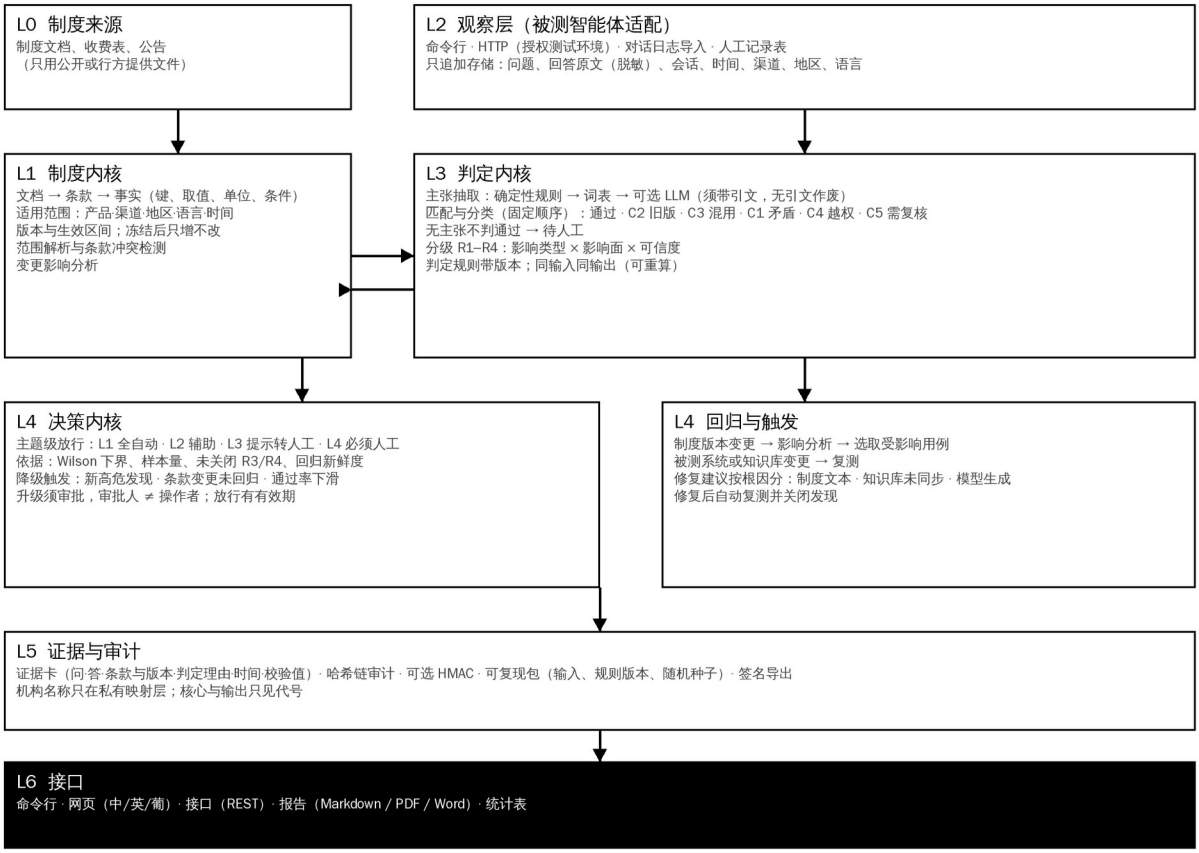
1　设计目标与八条不变量

答信不是“又一个质检打分器”。它要回答的是一个决策问题：这个主题能不能放给 AI 自动回答，依据是什么。因此架构围绕“结论可追溯、可重算、可审计”设计。以下八条不变量由代码强制，而不是靠约定：

#	不变量	怎么强制
I1	每个结论可追溯到（条款版本、回答原文、判定规则版本）三元组	Finding 必须携带三个引用，缺一不能入库
I2	观察与判定分离：观察只追加，判定可重算	Observation 表只允许 INSERT；Judgment 带 rule_version，规则升级后可整体重算
I3	抽不到主张不判通过	无主张→状态“待人工”，不进入通过分母
I4	标准是版本化事实，冻结后只增不改	Clause 状态机：草稿→已确认→冻结；冻结后修改只能生成新版本
I5	默认离线、确定性；LLM 可选，且不得单独决定通过或失败	LLM 输出须带条款引文，引文不在条款内即作废，降为“待人工”
I6	人在回路：定级与放行变更须审批，审批人≠操作者	审批动作写入审计链；同一用户同一对象不能既操作又审批
I7	核心只见机构代号，名称只在私有映射层	导出前做名称扫描，命中即拒绝输出
I8	统计诚实：比率必带样本量与区间；样本不足只降级不升级	放行引擎读取 n 与 Wilson 下界，不足则封顶在较低等级

2 分层架构

答信 核心架构（分层与数据流）



数据流：制度来源进入制度内核形成版本化事实；观察层把被测智能体的回答以只追加方式入库；判定内核把回答与事实比对得到发现与等级；决策内核把发现与统计变成主题级放行建议；证据与审计层固化每一步；接口层对外。

3 事件流（事件溯源）

系统以事件为唯一事实来源，状态是事件的投影。这使“重算”“回放”“审计”成为同一个机制。

事件	产生者	消费者
PolicyIngested / ClauseProposed	制度内核	人工确认界面
ClauseConfirmed / ClauseFrozen	审批人	判定内核、测试生成
TestSetGenerated	测试生成器	观察层
ObservationRecorded	观察层（适配器或人工记录）	判定内核
JudgmentComputed（带 rule_version）	判定内核	决策内核、证据层
HumanReviewed	复核人	判定内核（覆盖机器建议）、质量统计
FindingOpened / FindingClosed	判定内核、复测	放行引擎
ReleaseDecided	放行引擎 + 审批	报告、接口
RegressionTriggered	变更监听	观察层（重跑受影响用例）

4 核心组件设计

4.1 制度内核

- 对象：Source → Clause → Fact。Fact = （键、算符、取值、单位、条件）；键如 fee_waiver.min_purchases。
- 适用范围维度：产品、渠道、地区、语言、客户分层、时间区间。解析函数：resolve(key, scope, as_of) 返回该时点有效的事实。
- 优先级：范围更具体者优先；同等具体时生效日期晚者优先；同等具体且取值不同 = 条款冲突，生成独立发现（不是 AI 的错，是制度自身的问题）。
- 摄入流水线：文档解析 → 切分 → 规则（可选 LLM）提出事实候选 → 人工确认 → 冻结。候选一律不直接生效。
- 变更影响：新版本冻结时，计算受影响的主题、用例与已有放行，触发回归。

4.2 观察层

- 适配器接口：ask(question, context) → Answer{text, citations?, meta}；context 含日期、渠道、地区、语言、会话号。
- 接入形态：命令行进程、HTTP（仅授权测试环境）、对话日志导入（先脱敏）、人工记录表。对公开渠道只用人工记录表。
- 内置节流：HTTP 适配器带不可关闭的最小间隔与每日上限；超时或错误记为“无回答”，不计入通过。
- 存储：问题、回答原文（脱敏副本）、原文哈希、会话、时间、渠道、地区、语言。只追加。

4.3 主张抽取

三层，逐层兜底；每个主张记录来源层与置信度：

层	内容	说明
L1 确定性规则	数值、单位、比例、期限、有无与否定（“不收取”）、条件（“每年”）	按语言包实现：简体、繁体、英文、葡文；每个包有自己的测试集
L2 词表	越权承诺词（保证、一定、无风险）；转人工线索（转人工、客服热线）	词表带版本，可按地区与语言扩展
L3 可选 LLM	结构化抽取，输出必须符合模式并带原文引文	引文必须在回答文本内；否则丢弃。默认关闭

4.4 匹配与分类（固定顺序）

```
def classify(answer, scope, as_of, topic):
    claims = extract(answer)                        # 主张列表，可为空
    if any(promise(c) for c in claims): flags.add('C4')
    if topic.must_handoff:
        return PASS if handoff(answer) else 'C5'
    if not claims: return 'PENDING_HUMAN'          # I3：不判通过
    verdicts = []
    for c in claims:
        eff = resolve(c.key, scope, as_of)        # 当时有效的事实
        if c.matches(eff): verdicts.append(PASS)
        elif c.matches(any_older_version(c.key, scope)): verdicts.append('C2')
        elif c.matches(same_key_other_scope(c.key, scope)): verdicts.append('C3')
        else: verdicts.append('C1')
    return worst(verdicts) + flags                 # 答案级结论取最差，主张级结论全部保留
```

4.5 风险分级 R1–R4

输入：影响类型（金额与权益、期限与条件、一般信息）、影响面（该主题在用例或抽检中的占比，作代理指标并在报告中声明）、可信度（主张来源层与复核状态）。可信度低时只能标“待人工”，不直接定级。阈值为示例，须用试点数据校准。

4.6 放行引擎

```
def decide(topic):
    n, k = topic.sample_size, topic.errors          # 只计 C1–C4 的确认差异
    lb = wilson_lower(n - k, n)                      # 通过率的 95% 下界
    if topic.must_handoff: return 'L4'
    ceiling = 'L1'
    if open_findings(topic, {'R3','R4'}): ceiling = 'L3'          # 未关闭高危发现：封顶 L3
    if regression_stale(topic) or policy_changed_unverified(topic): ceiling =
lower(ceiling, 'L3')
    if n >= 73 and lb >= 0.95: level = 'L1'           # 零错误时 n≥73 才能到 95% 下界
    elif n >= 35 and lb >= 0.90: level = 'L2'
    elif lb >= 0.80: level = 'L3'
    else: level = 'L4'
    return lower(level, ceiling)                      # 取更保守的一级
```

- 升级须审批；放行有有效期，到期未复测自动降一级。
- 矩阵按主题×渠道×地区×语言分别计算，不合并成一个总分。

4.7 回归与变更触发

- 制度新版本冻结→影响分析→重跑受影响用例；被测系统或知识库声明变更→重跑核心集。
- 修复建议按根因分类：制度文本不清、知识库未同步、模型生成偏差；三类的处置与责任方不同。

4.8 证据与审计

- 证据卡：问、答（脱敏）、条款原文与版本、判定规则版本、判定理由、时间、校验值（链式 SHA-256），可选 HMAC。
- 可复现包：输入数据、条款版本、规则版本、随机种子、环境信息；别人拿到包能得到同样的判定。

4.9 测试生成与黄金集

- 生成：模板×槽位×改写包；多轮；诱导式用例单独成集，不计入错误率。覆盖度指标：每条款至少被几条用例覆盖，每个范围组合的样本量。
- 黄金集：两人独立标注，记录一致率与 kappa；用来估计每一类的精确率与漏报，并随规则版本回归。

4.10 验证器自身的质量

- 每个规则包有版本、测试集与发布说明；规则升级必须跑黄金集，精确率下降则不得发布。
- 报告里披露各类别精确率、待人工比例、情境不明比例。验证器自己也要“被验收”。

5 部署与安全

主题	设计
部署形态	单机容器或虚拟环境；SQLite（WAL）起步，规模增长后换 PostgreSQL；默认无外联，无遥测
角色	制度负责人、测试员、复核人、审批人、审计员；权限分离，审批人≠操作者
数据	对话先脱敏再入库；原文哈希入链；保留期可配置；删除时同步清除副本并留痕
密钥	HMAC 密钥与模型密钥只读环境变量，不写入文件与仓库
匿名	机构名称只在 private 映射；导出前扫描；命中则拒绝
离线安装	依赖固定版本并提供离线包；LLM 与网络功能默认关闭

6 接口

6.1 适配器与规则包 SPI

```
class Connector(Protocol):
    def ask(self, question: str, ctx: Context) -> Answer: ...      # 授权环境或导入的日志
class LanguagePack(Protocol):
    code: str                                                      # zh-Hans / zh-Hant /
en / pt
    def extract(self, text: str) -> list[Claim]: ...
    promise_terms: list[str]; handoff_cues: list[str]
class DomainPack(Protocol):
    topics: list[Topic]; must_handoff: set[str]; templates: list[Template]
```

6.2 命令行与接口

能力	命令行（暂定）	**REST**（暂定）
摄入制度	daxin policy ingest / confirm / freeze	POST /policies, POST /clauses/{id}/freeze
生成与导入用例	daxin tests generate / import	POST /tests
记录或运行观察	daxin observe record|run	POST /observations
判定与复核	daxin judge run / review	POST /judgments, POST /reviews
放行矩阵	daxin release show|decide	GET /release, POST /release/decisions
证据与审计	daxin evidence export / audit verify	GET /evidence/{id}, GET /audit
回归	daxin regress run	POST /regressions

7 与 Riffnel_memguard 的关系与代码组织

两条产品线共享一个“验证内核”：版本化事实、观察、判定、证据链。Riffnel 面向“智能体记忆”，答信面向“客服回答”。建议把共同部分抽成内核包，两边各自加领域层，避免两份重复的引擎。

内核能力	**Riffnel** 现有代码	处理方式
条款与版本	core/ models.py（ClauseVersion、texts、units）	抽为内核；加适用范围解析与冲突检测
主张抽取	extract/rules.py（中/葡/英数字与单位）	拆成语言包；加金额、比例、否定、条件
调查与分类	engines/investigator.py、linker.py	答信的 classify 复用其“当时有效版本”逻辑；记忆归因部分留在 Riffnel
跨语言一致性	engines/crosslingual.py	直接复用，作为答信的“语言”测试方法
修正与回滚	engines/corrector.py	答信只借“计划、审批、日志”的状态机；对象改为知识库条目
验证与回归	engines/verifier.py	复用重放与回归；加主题级统计
审计与证据	core/audit.py、report/markdown.py	复用哈希链与 HMAC；证据卡新增模板
界面与国际化	ui/app.py、i18n.py	复用框架；答信新增中文页面
智能体协议	agent/command.py	作为观察层的命令行适配器

组织方式：先在独立仓库做答信（依赖 riffnel-memguard），比赛结束后再把稳定部分上移为内核包。不要在赛前做大重构。

8 分阶段实现与验收

阶段	范围	验收标准
M0（第 1 周）	制度内核（条款、事实、版本、冻结）；观察层记录表导入；匿名防护	导入示例制度并可按时点解析；记录表可导入；含名称的输出被拒绝
M1（第 2 周）	主张抽取（中文 + 金额、比例、否定）；分类 C1–C5；R1–R4	黄金集上各类精确率可计算；无主张不判通过有测试

M2（第 3 周）	放行引擎；回归触发；证据卡；中文界面	端到端：发现→证据卡→修复→复测→矩阵变化
M3（第 4 周）	语言包（繁体、英文、葡文）；统计与报告；演示与文档	在干净环境按文档从头跑通；全部数字来自实际输出
V1（赛后 3 个月）	HTTP 适配器（授权环境）、PostgreSQL、REST、规则包版本化	试点环境内跑一个场景

9 关键设计取舍与理由

取舍	选择	理由
规则优先 vs LLM 裁判	规则优先，LLM 只能辅助且必须带引文	银行场景要可复现、可解释；LLM 判决无法稳定复现，也难以对外举证
黑盒旁路 vs 深度集成	黑盒旁路	外部工具难以进入银行内部体系；旁路也更容易获批试点
总分 vs 主题级放行	主题级，且不合并总分	放行是按主题做的决定；总分会掩盖局部高风险
自动升级 vs 审批	升级须审批，降级可自动	误放行代价高于误降级
全量 vs 抽样	测试集全量跑，线上用抽检	测试集可重复；线上抽检估计漏报，二者互补
统一模型 vs 语言包	语言包插件	每种语言的数字、否定、量词写法不同，需要各自的测试集

10 限制与待验证

- 规则抽取只覆盖有限句式；复杂条件、多重否定、表格式回答可能漏判，漏判一律落到“待人工”，不落到“通过”。
- 影响面用主题占比作代理，不是真实流量；报告中必须声明。
- 阈值（R1–R4、L1–L4、置信水平）为示例，需用试点数据与银行意见校准。
- 目前没有任何真实机构的数据；所有结论来自仿真或公开文件的人工核对。
- 本系统提供一致性检查与证据，不构成法律意见，不替代法务、合规与消保的判断。